



清华大学

Tsinghua University

习题解析

《计算机网络原理》教学团队

2025年12月



Wireshark实验4——TCP/UDP

- UDP数据包在IP层的类型编号是？ 17
- UDP数据包头字段依次是？
 - Src Port, Dst Port, Length, Checksum
- TCP数据包在IP层的类型编号是？ 6
- TCP数据包头字段依次是？
 - 如图，注意有很多是Wireshark解析的内容，并非包头字段

```
Internet Protocol Version 4, Src: 30.246.181.103, Dst: 30.30.30.30
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 62
    Identification: 0x4ca4 (19620)
  > 000. .... = Flags: 0x0
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: UDP (17)
    Header Checksum: 0xd72 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 30.246.181.103
    Destination Address: 30.30.30.30
    [Stream index: 4]
User Datagram Protocol, Src Port: 61454, Dst Port: 53
  Source Port: 61454
  Destination Port: 53
  Length: 42
  Checksum: 0x9712 [unverified]
  [Checksum Status: Unverified]
  [Stream index: 0]
  [Stream Packet Number: 1]
  > [Timestamps]
  UDP payload (34 bytes)
```

UDP

```
Internet Protocol Version 4, Src: 30.246.181.103, Dst: 59.110.73.151
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 40
    Identification: 0x0000 (0)
  > 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: TCP (6)
    Header Checksum: 0xe16d [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 30.246.181.103
    Destination Address: 59.110.73.151
    [Stream index: 3]
Transmission Control Protocol, Src Port: 62973, Dst Port: 443, Seq: 387, Ack: 5437, Len: 0
  Source Port: 62973
  Destination Port: 443
  [Stream index: 6]
  [Stream Packet Number: 17]
  [Conversation completeness: Complete, WITH_DATA (31)]
  [TCP Segment Len: 0]
  Sequence Number: 387 (relative sequence number)
  Sequence Number (raw): 1672895189
  [Next Sequence Number: 387 (relative sequence number)]
  Acknowledgment Number: 5437 (relative ack number)
  Acknowledgment number (raw): 2869125630
  0101 .... = Header Length: 20 bytes (5)
  > Flags: 0x010 (ACK)
    Window: 4095
    [Calculated window size: 262080]
    [Window size scaling factor: 64]
    Checksum: 0x832c [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
```

TCP



Wireshark实验4——TCP/UDP

- TCP三次握手过程使用三个数据包，他们的标记位，序列号，确认序列号有什么特点？TCP握手时使用选项协商链接参数，举出一个例子？
 - $\text{packet1.seq} = x, \text{packet2.seq} = y, \text{packet2.ack} = x + 1, \text{packet3.seq} = x+1, \text{packet3.ack} = y + 1$ ；比如图中两者都支持SACK，则会使用SACK
- TCP传输过程中利用序列号和确认序列号实现数据的可靠传输。序列号增长和包长关系是什么？确认包确认序列号和原包序列号的关系是什么？
 - 序列号增长上一个包的TCP payload长度（IPv4时 = IP payload len – IP header len – TCP header len），确认包序列号等于他所确认的包的序列号 + TCP payload长度

```
62965 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=2177974368 TSecr=0 SACK_PERM
443 → 62965 [SYN, ACK] Seq=0 Ack=1 Win=42340 Len=0 MSS=1440 SACK_PERM WS=512
62965 → 443 [ACK] Seq=1 Ack=1 Win=262144 Len=0
```

三次握手

```
54 62965 → 443 [ACK] Seq=423 Ack=4736 Win=261824 Len=0
953 Application Data
60 443 → 62965 [ACK] Seq=4736 Ack=1322 Win=42496 Len=0
```

前两个是同一方向的包，
最后一个反向的

中间包的
序列号，长
度信息

```
Transmission Control Protocol, Src Port: 62965, Dst Port: 443, Seq: 423, Ack: 4736, Len: 899
Source Port: 62965
Destination Port: 443
[Stream index: 2]
[Stream Packet Number: 16]
> [Conversation completeness: Complete, WITH_DATA (31)]
[TCP Segment Len: 899]
Sequence Number: 423 (relative sequence number)
Sequence Number (raw): 1032741626
[Next Sequence Number: 1322 (relative sequence number)]
Acknowledgment Number: 4736 (relative ack number)
Acknowledgment number (raw): 3725792088
0101 .... = Header Length: 20 bytes (5)
> Flags: 0x018 (PSH, ACK)
Window: 4096
[Calculated window size: 262144]
[Window size scaling factor: 64]
Checksum: 0x17fe [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
< [Timestamps]
[Time since first frame in this TCP stream: 0.039363000 seconds]
```

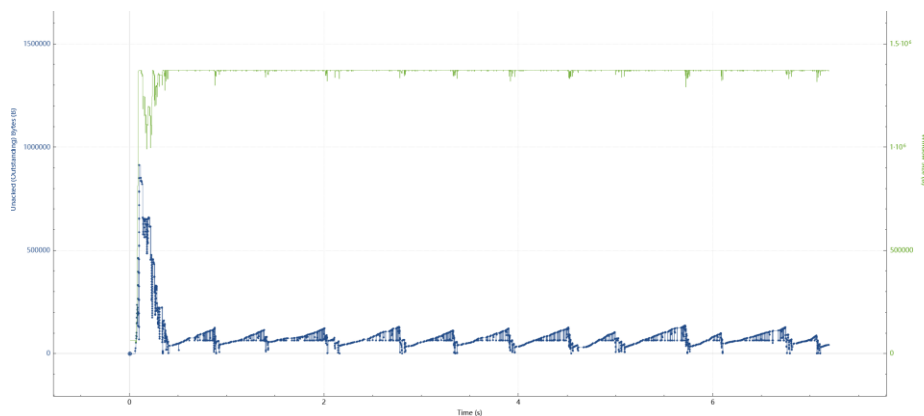


Wireshark实验4——TCP/UDP

- TCP四次挥手过程使用四个数据包，他们的标记位，序列号，确认序列号有什么特点？（选做）
 - Packet1: fin = 1, ack = x, seq = y; Packet2: fin = 0, ack = y + 1, seq = x; Packet3: fin = 1, seq = x, ack = y + 1; Packet4: fin = 0, seq = y + 1, ack = x + 1
- 使用Wireshark工具栏的统计->TCP流图形功能，观察TCP拥塞控制算法的行为。（选做）
 - Iperf打TCP流，用tcpdump或wireshark进行抓包，观察到明显的周期性窗口变化，图中用的Cubic进行测试，能一定程度看到接近三次函数的曲线变化

```
54 62978 → 80 [FIN, ACK] Seq=53 Ack=30528 Win=262144 Len=0
60 80 → 62978 [ACK] Seq=30528 Ack=54 Win=78848 Len=0
60 80 → 62978 [FIN, ACK] Seq=30528 Ack=54 Win=78848 Len=0
54 62978 → 80 [ACK] Seq=54 Ack=30529 Win=262144 Len=0
```

四次挥手



窗口变化



Wireshark实验4——思考题

- TCP建立连接时使用选项协商MTU信息。上网查资料，TCP选项还支持什么特殊的功能？
 - SACK, Timestamp, Window Scale等
- 反射DoS攻击中，攻击者将数据包源地址改为受害者IP向公共服务(DNS, NTP等等)发送请求，公共服务回复数据包至受害者IP，使受害者带宽耗尽。为什么此类攻击大多使用基于UDP的公共服务，而不是基于TCP呢？
 - TCP需要三次握手建立连接，受害者收到第二个握手包（SYN, ACK）后发现自己没有发过第一个握手包（SYN），于是不会选择建立连接，后续数据不会发送，没有起到流量放大的作用



Wireshark实验5——HTTP(s)

- HTTP使用的传输层协议是什么？
 - TCP
- HTTP请求包请求的方法是什么？请求的Host和URL是什么？使用的HTTP版本是什么？
 - GET, Host: example.com, URL: http://example.com, HTTP/1.1
- HTTP响应包的状态码是什么？响应的Content-Type是什么？
 - 200, text/html

```
> Ethernet II, Src: Intel_85:b2:45 (bc:38:98:85:b2)
> Internet Protocol Version 4, Src: 183.172.73.23,
> Transmission Control Protocol, Src Port: 50116,
✓ Hypertext Transfer Protocol
  > GET / HTTP/1.1\r\n
    Host: example.com\r\n
    User-Agent: curl/8.9.1\r\n
    Accept: */*\r\n
    \r\n
    [Response in frame: 17424]
    [Full request URI: http://example.com/]
```

```
> Transmission Control Protocol, Src Port: 80, Dst Port: 50116, Seq. ...
> [2 Reassembled TCP Segments (827 bytes): #17423(822), #17424(5)]
✓ Hypertext Transfer Protocol, has 2 chunks (including last chunk)
  > HTTP/1.1 200 OK\r\n
    Date: Sun, 11 Jan 2026 09:05:33 GMT\r\n
    Content-Type: text/html\r\n
    Transfer-Encoding: chunked\r\n
    Connection: keep-alive\r\n
    CF-RAY: 9bc340c9a9baa29d-HKG\r\n
    Last-Modified: Sat, 03 Jan 2026 05:43:21 GMT\r\n
    Allow: GET, HEAD\r\n
    Accept-Ranges: bytes\r\n
    Age: 1884\r\n
    cf-cache-status: HIT\r\n
    Server: cloudflare\r\n
    \r\n
    [Request in frame: 17421]
    [Time since request: 0.045773000 seconds]
    [Request URI: /]
    [Full request URI: http://example.com/]
  > HTTP chunked response
    File Data: 513 bytes
> Line-based text data: text/html (1 lines)
```



Wireshark实验5-HTTP(s)

- TLS数据包使用的TLS版本是什么？你还能看到HTTP请求或响应的内容吗？为什么？
 - TLS1.3, 不能，经过加密了
- 简要展示下HTTPS整体的交互流程
 - Client先发送client hello，携带自身支持的协议，选项，密钥交换参数等信息，然后server回复server hello， change cipher spec，确认参数，计算出密钥，通知后续消息都使用加密，客户端同样回复change cipher spec
- 感兴趣的同学可以尝试参照以下链接进行HTTPS解密，观察解密后访问Deepseek的包的内容

- 图中标记为HTTP的包实际为解密后的https包

92	4.155018	183.172.73.23	60.204.2.5	TLSv1.3	1845 Client Hello (SNI=chat.deepseek.com)
95	4.192026	60.204.2.5	183.172.73.23	TLSv1.3	1394 Server Hello, Change Cipher Spec, Application Data
101	4.193595	60.204.2.5	183.172.73.23	TLSv1.3	450 Application Data, Application Data, Application Data
104	4.194764	183.172.73.23	60.204.2.5	TLSv1.3	134 Change Cipher Spec, Application Data

25	5.476864	183.172.73.23	60.204.2.5	TLSv1.3	82 Application Data
69	5.388868	183.172.73.23	60.204.2.5	TLSv1.3	1781 Client Hello (SNI=chat.deepseek.com)
73	5.433611	60.204.2.5	183.172.73.23	TLSv1.3	1394 Server Hello, Change Cipher Spec, Encrypted Extensions
77	5.433611	60.204.2.5	183.172.73.23	TLSv1.3	450 Certificate, Certificate Verify, Finished
80	5.435948	183.172.73.23	60.204.2.5	TLSv1.3	134 Change Cipher Spec, Finished
81	5.436242	183.172.73.23	60.204.2.5	HTTP	962 GET / HTTP/1.1
84	5.477060	60.204.2.5	183.172.73.23	TLSv1.3	357 New Session Ticket
85	5.477060	60.204.2.5	183.172.73.23	TLSv1.3	357 New Session Ticket
87	5.495493	60.204.2.5	183.172.73.23	TCP	1394 443 -> 65174 [ACK] Seq=5099 Ack=2716 Win=35840 Len=1340 [TCP PDU reassembled]
88	5.495493	60.204.2.5	183.172.73.23	TLSv1.3	1394 Server Hello, Change Cipher Spec, Encrypted Extensions
89	5.495493	60.204.2.5	183.172.73.23	TLSv1.3	450 Certificate, Certificate Verify, Finished
90	5.495493	60.204.2.5	183.172.73.23	TLSv1.3	134 Change Cipher Spec, Finished
110	5.534363	183.172.73.23	121.194.10.213	TLSv1.3	1815 Client Hello (SNI=static.deepseek.com)
113	5.535606	183.172.73.23	121.194.10.213	TLSv1.3	1815 Client Hello (SNI=static.deepseek.com)
118	5.543709	121.194.10.213	183.172.73.23	TLSv1.3	1394 Server Hello, Change Cipher Spec, Encrypted Extensions
122	5.543709	121.194.10.213	183.172.73.23	TLSv1.3	1394 Server Hello, Change Cipher Spec, Encrypted Extensions
126	5.543709	121.194.10.213	183.172.73.23	TLSv1.3	919 Certificate, Certificate Verify, Finished

Frame 81: 962 bytes on wire (7696 bits), 962 bytes captured (7696 bits) on interface \Device\NPF...{29A043BC-A660-451...}

Ethernet II, Src: Intel_R51b2:45 (bc38:98:85:b2:45), Dst: IETF_VRRP_VRID_01 (00:00:5e:00:01:01)

Internet Protocol Version 4, Src: 183.172.73.23, Dst: 60.204.2.5

Transmission Control Protocol, Src Port: 65174, Dst Port: 443, Seq: 1808, Ack: 4493, Len: 908

Transport Layer Security

TLSv1.3 Record Layer: Application Data Protocol: Hypertext Transfer Protocol

Opaque Type: Application Data (23)

Encrypted Application Data [-]: e38f9745e927257d8b883e3b5f4b46c12359cc52b74be1377cb347962cc78400ece0582526a5

[Application Data Protocol: Hypertext Transfer Protocol]

Hypertext Transfer Protocol

GET / HTTP/1.1\r\n

Host: chat.deepseek.com\r\n

Connection: keep-alive\r\n

sec-ch-ua: "Google Chrome" ;v="143", "Chromium" ;v="143", "Not A(Brand)" ;v="24" ;v="143"\r\n

sec-ch-ua-mobile: ?0\r\n

sec-ch-ua-platform: "Windows"\r\n

Upgrade-Insecure-Requests: 1\r\n\r\n

User-Agent: Mozilla/5.0 (Windows ; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36\r\n

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png; */*;q=0.8,application/signed-exchange;v=b3;q=0.0\r\n

Sec-Fetch-Site: none\r\n

Sec-Fetch-Mode: navigate\r\n

Sec-Fetch-User: ?1\r\n

Sec-Fetch-Dst: document\r\n

Accept-Encoding: gzip, deflate, br, zstd\r\n

Accept-Language: zh-CN,zh;q=0.9\r\n

Cookie: smidv2=20260111719405e77fc54d2be09c345f1e5a2f5e589b005124d81d16237f0; .thumbcache_6b2e5483f9d858d7c661\r\n\r\n

[Response in frame: 90]

[Full request URI: https://chat.deepseek.com/]



Wireshark实验5——思考题

- 根据实验观察到的结果和课程内容，分析HTTP协议的头部与IP头或TCP头的设计思路差异
 - IP或TCP包头更追求处理效率，因此包头长度一般都是固定的，有额外的信息需要通过选项提供，编码效率很高，一字节有时携带很多信息。Http包头则更加关注信息可读性，编码效率较低，采用ascii码。



清华大学

Tsinghua University

谢谢